

INF333 2024-2025 Spring Semester

Homework II Design Document

Please provide answers inline in a `quote` environment.

1 Preliminaries

Q1: If you have any preliminary comments on your submission, notes for the TAs, or extra credit, please give them here.

Q2: Please cite any offline or online sources you consulted while preparing your submission, other than the Pintos documentation, course text, and lecture notes.

- <https://www.youtube.com/watch?v=6pLAybZ2yxo>
- https://www.youtube.com/watch?v=pPzVV2kkGHc&list=PLF2K2xZjNEf97A_uBCwEl61sdxWVP7VWC

2 Argument Passing

2.1 Data Structures

Q3: Copy here the declaration of each new or changed ‘struct’ or ‘struct’ member, global or static variable, ‘typedef’, or enumeration.

Identify the purpose of each in 25 words or less.

Declaration	Purpose (25 words)
<code>typedef int pid_t;</code>	Type alias for process identifiers; improves code readability and portability.
<code>struct write_request</code>	Represents a pipe write request with a message, ticket, and name for matching.
<code>struct read_request</code>	Represents a pipe read request, includes semaphore for synchronization and name matching.
<code>static struct list write_requests;</code>	Global list storing pending write requests in pipe simulation.
<code>static struct list read_requests;</code>	Global list storing pending read requests in pipe simulation.
<code>struct process_identifier</code>	Holds a process ID and list element to track child processes.
<code>struct fd_entry</code>	Represents an open file entry with file descriptor and pointer to the file.
<code>static int (*syscall_handlers[20])(struct intr_frame *);</code>	Array of function pointers indexed by syscall number to dispatch correct handler.

Tab. 1: New or modified structs, typedefs, and globals with brief descriptions.

2.2 Algorithms

Q4: Briefly describe how you implemented argument parsing. How do you arrange for the elements of `argv[]` to be in the right order? How do you avoid overflowing the stack page?

We tokenize the command-line string using `strtok_r` and store each token in an array (`argv[]`) while counting them in `argc`. This splits the input cleanly and prevents too many tokens from being created.

In `setup_stack`, we push the argument strings onto the stack in reverse order. We record their addresses and then push those addresses (again in reverse order) so that when read in proper memory order, they appear as originally typed.

A single, fixed-size stack page is allocated, and strict pointer arithmetic and alignment methods ensure we never push more data than the page can hold. Maximum input sizes (like `ARGUMENT_MAX`) also prevent overflow.

2.3 Rationale

Q5: Why does Pintos implement `strtok_r()` but not `strtok()`?

The key difference between `strtok_r()` and `strtok()` is that in `strtok_r()` we must provide our own pointer (called `save_ptr`) to remember the position in the string. In Pintos, the kernel divides commands into a command name and arguments, and we need a reliable way to store the position of these arguments. By using `strtok_r()`, each thread gets its own separate pointer, allowing multiple threads to safely parse strings at the same time without conflicts.

Q6: In Pintos, the kernel separates commands into an executable name and arguments. In Unix-like systems, the shell does this separation. Identify at least two advantages of the Unix approach.

1. It's simpler and clearer to split the program name and its arguments before giving them to the kernel, since they're different things. Parsing commands shouldn't be the kernel's job—it can easily be done by a user program instead.
2. Checking user input could also be safer if handled by the shell instead of the kernel. For example, if someone enters too much text, it could cause problems for the kernel. But if the shell handles it, the worst that could happen is just the shell crashes, not the whole system.
3. The shell can also separate commands and do advanced preparation or checking. This means it can behave more like a smart interpreter, processing multiple commands at once, rather than simply passing input directly to the kernel.

3 System Calls

3.1 Data Structures

Q7: Copy here the declaration of each new or changed struct or struct member, global or static variable, typedef, or enumeration. Identify the purpose of each in 25 words or less.

Declaration	Purpose (25 words)
typedef int pid_t;	Type alias for process identifiers; improves code readability and portability.
struct write_request	Represents a pipe write request with a message, ticket, and name for matching.
struct read_request	Represents a pipe read request, includes semaphore for synchronization and name matching.
static struct list write_requests;	Global list storing pending write requests in pipe simulation.
static struct list read_requests;	Global list storing pending read requests in pipe simulation.
struct process_identifier	Holds a process ID and list element to track child processes.
struct fd_entry	Represents an open file entry with file descriptor and pointer to the file.
static int (*syscall_handlers[20])(struct intr_frame *);	Array of function pointers indexed by syscall number to dispatch correct handler.

Q8: Describe how file descriptors are associated with open files. Are file descriptors unique within the entire OS or just within a single process?

We assign unique file descriptors within each process by incrementing a counter stored in the thread structure whenever a file is opened. For each opened file, we create a new **process_fd** struct and add it to the list of file descriptors maintained by the thread. We also update a global hash table that keeps track of all open files.

Q9: Describe the sequence of events when **lock_release()** is called on a lock that a higher-priority thread is waiting for.

When **lock_release** is called, the current thread gives up the lock. If another thread is waiting for the lock, the one with the highest priority is selected. That thread is removed from the waiting queue and marked as ready to run. The releasing thread's priority is updated, usually back to its original base priority. If the unblocked thread has higher priority, the CPU may switch to it immediately.

3.2 Algorithms

Q10: Describe your code for reading and writing user data from the kernel.

In Pintos, when a user program calls functions like `read()` or `write()`, it gives a pointer to a buffer in user memory. Since the kernel runs in a different (privileged) memory space, it cannot trust these pointers directly.

To avoid crashing the system:

- We check if the given pointer really belongs to the user space.
- If the pointer is invalid or points to kernel memory, we immediately terminate the user program.
- For strings, we check that the string is readable and null-terminated.

We do all these checks before actually reading or writing any data. If everything looks good, we safely access the memory and either copy data into it (for `read()`) or take data out of it (for `write()`).

This helps the kernel avoid bugs or crashes caused by bad memory access.

Q11: Suppose a system call causes a full page (4,096 bytes) of data to be copied from user space into the kernel. What is the least and the greatest possible number of inspections of the page table (e.g. calls to `pagedir_get_page()`) that might result? What about for a system call that only copies 2 bytes of data? Is there room for improvement in these numbers, and how much?

A simple approach would be to check each byte individually to ensure it is on a valid page. However, since a page is always 4096 bytes, any piece of data can span at most two pages. Therefore, we only need to perform this check at most twice—once if the data starts exactly at a page boundary, or twice if it crosses between two pages.

Q12: Briefly describe your implementation of the "wait" system call and how it interacts with process termination.

The `wait` system call is implemented in `process_wait()`. It allows a parent process to wait for a specific child process (identified by its `pid`) to terminate. Before blocking, the function checks whether the requested child PID actually belongs to the calling process using a helper function `process_is_parent_of()`. To avoid duplicate waiting on the same child, the PID is removed from the `childThreads` list right after validation.

The wait mechanism uses a communication primitive, `connector_read()`, which blocks the caller until the corresponding `connector_write()` is triggered by the child process in `process_exit()`. When a process exits, it calls `connector_write()` with its PID and status code, which unblocks the waiting parent process and delivers the exit status.

This approach ensures proper parent-child synchronization and guarantees that the exit status is returned correctly, while also preventing memory leaks by cleaning up child records immediately.

Q13: Any access to user program memory at a user-specified address can fail due to a bad pointer value. Such accesses must cause the process to be terminated. System calls are fraught with such accesses, e.g. a "write" system call requires reading the system call number from the user stack, then each of the call's three arguments, then an arbitrary amount of user memory, and any of these can fail at any point. This poses a design and error-handling problem: how do you best avoid obscuring the primary function of code in a morass of error-handling? Furthermore, when an error is detected, how do you ensure that all temporarily allocated resources (locks, buffers, etc.) are freed? In a few paragraphs, describe the strategy or strategies you adopted for managing these issues. Give an example.

I did not understand the question clearly

3.3 Synchronization

Q14: The "exec" system call returns -1 if loading the new executable fails, so it cannot return before the new executable has completed loading. How does your code ensure this? How is the load success/failure status passed back to the thread that calls "exec"?

In our implementation, the `exec` system call spawns a new thread via `thread_create()` which runs `start_process()`. However, `process_execute()` must not return until the new process finishes loading the executable.

To ensure this, we use a message-passing mechanism between the parent and child threads, implemented with `connector_write()` and `connector_read()`. The child process calls `connector_write("exec", tid, -1)` if loading fails, or `connector_write("exec", tid, tid)` if it succeeds.

Meanwhile, the parent blocks using `connector_read("exec", tid)` right after creating the thread. This causes it to wait until the child thread explicitly signals success or failure. This guarantees that the `exec()` system call returns only after loading is finished, and returns -1 if it fails.

Q15: Consider parent process P with child process C. How do you ensure proper synchronization and avoid race conditions when P calls `wait(C)` before C exits? After C exits? How do you ensure that all resources are freed in each case? How about when P terminates without waiting, before C exits? After C exits? Are there any special cases?

In Pintos, when a parent creates a child process, synchronization is managed using the connector system with functions like `connector_read` and `connector_write`.

If the parent calls `wait` before the child exits, the parent will block and wait. The child writes its exit status when it finishes, and this wakes up the parent.

This avoids race conditions because the parent cannot go ahead without the child's result.

If the child exits before the parent calls `wait`, the status is already written and stored. Later, when the parent finally calls `wait`, it reads the status directly and continues.

If the parent dies before calling `wait`, the child still writes its status, but no one reads it. This doesn't cause problems, because all resources of the parent and child are still cleaned up.

If the parent calls `wait` twice for the same child, the second call will fail. Also, the parent cannot wait for a grandchild process.

In all cases, memory and file descriptors are cleaned up during `process_exit`. This ensures there are no memory leaks.

3.4 Rationale

Q16: Why did you choose to implement access to user memory from the kernel in the way that you did?

In my implementation, I created two helper functions: `read_user_byte()` and `write_user_byte()` to safely read and write single bytes from/to user virtual addresses. These use inline assembly to attempt access, but safely recover if a fault occurs. I also added utility checks like `is_valid_pointer()` and `is_valid_string()` before using any user-supplied pointer or string in a system call.

I chose this method because it protects the kernel from invalid user memory access, which can cause crashes or security issues. Instead of assuming a pointer is valid, I proactively validate it byte by byte. This keeps the OS stable, even when the user process passes a bad or malicious pointer.

Additionally, I avoided page fault handling inside the kernel because virtual memory is not fully implemented yet. So I exit the thread when unsafe access is detected.

This approach is modular, secure, and easy to test. By isolating unsafe operations into specific helper functions, I reduced the risk of bugs and improved maintainability.

Q17: What advantages or disadvantages can you see to your design for file descriptors?

The file descriptor design in our system gives each process its own descriptor table. This prevents interference between processes, making it safer and more modular. File descriptors start from 2, reserving 0 and 1 for standard input and output. This aligns with UNIX-like conventions and avoids conflicts. The `fd_entry` struct maps each descriptor to a file, making access efficient. Also, since each process tracks its open files, we can easily close all of them on exit. However, this design has limitations. File descriptors are linear and not reused efficiently. If a process opens and closes many files, gaps in descriptor numbers may grow. Also, memory is used per open file because each `fd_entry` is dynamically allocated. This might become costly for processes that manage many files.

Overall, the design is clean and simple to implement, but could be improved for memory efficiency and better descriptor reuse.

Q18: The default `tid_t` to `pid_t` mapping is the identity mapping. If you changed it, what advantages are there to your approach?

Our system gives each process its own table of file descriptors. This keeps processes separate and organized. File descriptors start at 2 because 0 and 1 are reserved for standard input and output, just like in UNIX systems. Each file descriptor is connected to a file using the `fd_entry` struct, which makes file access quick. Also, keeping track of open files lets us easily close them when the process ends.

But there are some downsides. File descriptors are given out in increasing order and aren't reused effectively, which can create gaps if files are frequently opened and closed. Additionally, each `fd_entry` takes up memory, so opening many files can use up a lot of resources.

Overall, our current approach is easy to understand and implement, but could be improved to better reuse descriptors and save memory.

4 Survey Questions

Answering these questions is optional, but it will help us improve the course in future quarters. Feel free to tell us anything you want—these questions are just to spur your thoughts. You may also choose to respond anonymously in the course evaluations at the end of the quarter.

Q1: In your opinion, was this assignment, or any one of the three problems in it, too easy or too hard? Did it take too long or too little time?

Answer here

Q2: Did you find that working on a particular part of the assignment gave you greater insight into some aspect of OS design?

Answer here

Q3: Is there some particular fact or hint we should give students in future quarters to help them solve the problems? Conversely, did you find any of our guidance to be misleading?

Answer here

Q4: Do you have any suggestions for us to more effectively assist students, either for future semesters or the remaining projects?

Answer here

Q5: Any other comments?

Answer here